



SECURITY AUDIT

Parallel

sPRLV2

FINAL REPORT

June 2026

BAILSEC.IO



Disclaimer:

Security assessment projects are time-boxed and reliant on information provided by the client, its affiliates, or its partners. The findings in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase, and apply only to the specific commit hash, version, or deployment reviewed at the time of the audit. Any subsequent modification, redeployment, or upgrade is outside the scope of this report and may invalidate its findings.

The audit does not constitute investment, legal, financial, regulatory, or tax advice, nor an endorsement of any project or team. It is based on a limited review of materials provided and is delivered on an "as-is," "where-is," and "as-available" basis. Use of blockchain technology is subject to unknown risks and flaws. The scope of this assessment is technical and contract-level; unless explicitly stated otherwise, it does not cover economic or game-theoretic risks, MEV, oracle manipulation, governance attacks, centralization or admin-key risks, off-chain components, front-end code, or the behavior of third-party protocols, dependencies, libraries, or services interacted with by the audited code.

We make no warranties, express or implied, regarding the accuracy, reliability, completeness, or availability of the report or any associated services, and disclaim all implied warranties including merchantability, fitness for a particular purpose, and non-infringement. We assume no responsibility for any product, service, software, code, or material referenced by, linked to, or accessible through the report.

This report is prepared solely for the contract owner and may not be relied upon by any third party. The contract owner is responsible for making their own decisions based on the report and should seek additional professional advice as needed. To the maximum extent permitted by law, our total aggregate liability arising from or related to this audit shall not exceed the fees actually paid for the engagement. The contract owner agrees to indemnify and hold harmless the audit firm and its personnel from any claims, damages, expenses, or liabilities arising from use of or reliance on the report or the audited smart contract.

This disclaimer and any dispute arising from the audit shall be governed by the laws of Wyoming (USA), without regard to its conflict-of-laws principles. By engaging in this audit, the contract owner acknowledges and agrees to these terms.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Parallel - sPRLV2 – Audit Report
Website	app.parallel.best
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/parallel-protocol/parallel-tokenomics/blob/9a7a30b314020df99fb156306d7c431cb4a54e00/contracts/sPRL/
Resolution 1	

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium	2					
Low	5					
Informational	9					
Governance						
Total	16					

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

TimelockPenaltyERC20

The **TimelockPenaltyERC20** contract is an abstract contract which is inherited by the **sPRL2V2** contract and it implements asynchronous withdrawals where users first need to request a withdrawal via **requestWithdraw** which sets the beginning time of the withdrawal request and the end time of the withdrawal request. Users have then three options:

- a: Cancel the request. This will re-mint the burned shares.
- b: Wait until the deadline has passed. This will then allow for withdrawal without a penalty
- c: Finalize the request earlier. This will result in a penalty which is taken as BPT tokens and transferred to the feeReceiver.

Appendix: Fee Calculation

The early-exit slash applied on **withdrawBPT** / **withdrawPRLAndWeth** is computed by

```
> _calculateFee[_amount, _requestTime, _releaseTime]
```

For a request that has not yet matured ($\text{block.timestamp} < \text{_releaseTime}$), the contract takes

```
> timeLeft = _releaseTime - block.timestamp
```

and

the per-request

```
> lockDuration = _releaseTime - _requestTime [frozen at request time, equal to the  
timeLockDuration then in effect].
```

Then derives a linearly-decaying penalty rate:

```
> feePercentage = startPenaltyPercentage.mulDivUp(timeLeft, lockDuration)
```

and the slash:

```
> feeAmount = _amount.wadMulUp[feePercentage] [1e18-scaled WAD].
```

The penalty is thus at its maximum (`startPenaltyPercentage`) at request time and decays linearly to zero as `_releaseTime` approaches; once `block.timestamp >= _releaseTime` the function short-circuits to 0, so a fully matured withdrawal is slash-free.

Because the live `startPenaltyPercentage` is bounded to `MAX_PENALTY_PERCENTAGE` ($1e18 = 100\%$) and `timeLeft <= lockDuration`, the result satisfies `feePercentage <= startPenaltyPercentage <= 1e18` and therefore `feeAmount <= _amount` (no over-slash).

Both `mulDivUp` and `wadMulUp` round up, so the slash rounds in the protocol's favor: the `feeReceiver` receives `feeAmount` and the user receives `_amount - feeAmount`.

Privileged Functions

- `updateTimelockDuration`
- `updateStartPenaltyPercentage`
- `updateFeeReceiver`
- `pause`
- `unpause`

Invariants

INV 1: Users can request and cancel withdrawals at all time

INV 2: `unlockingAmount` must always reflect the current requested and not-yet-claimed amount

INV 3: The fee must decrease the more time has passed after a withdrawal request

INV 4: The withdrawal fee must be rounded up

Issue_01	Shared nonce can break assumptions
Severity	Low
Description	TimeLockPenaltyERC20, the base of both sPRL2 and sPRL2V2, is simultaneously an ERC20Permit (EIP-2612 gasless approvals) and an ERC20Votes token (gasless vote delegation via delegateBySig). Both of those signed-message entry points draw replay protection from the same OpenZeppelin Nonces base, and the contract's nonces[address] resolver at TimeLockPenaltyERC20.sol:392 [override(ERC20Permit, Nonces)] exists precisely because both parents bring Nonces and must be reconciled to a single per-owner counter. These flows matter because the sToken is a user-facing instrument: a depositor may use permit to grant the migrator (or any spender) a gasless sPRL2 allowance, and a holder may use delegateBySig to delegate governance weight gaslessly – both legitimate, reachable actions on the same token and the same owner counter. Because permit and delegateBySig consume the one shared Nonces._nonces[owner] counter, the two signature types are not independent. permit does not take an explicit nonce parameter, it recomputes the EIP-2612 digest using _useNonce(owner), i.e. the owner's current on-chain nonce at execution; delegateBySig takes an explicit nonce and enforces it via _useCheckedNonce(signer, nonce). If a user holds two outstanding signatures created against the same nonce value, one permit, one delegateBySig, only the first submitted can succeed: executing either advances the counter, after which the other reverts (permit recovers a signer that no longer equals owner → ERC2612InvalidSigner; delegateBySig finds nonce != current → InvalidAccountNonce). The issue is reachable because both entrypoints are permissionless, so the submitter (a relayer, a mempool observer, or the user's own two transactions in arbitrary order) decides which signature consumes the nonce first.

	Impact 1: A user's pre-signed permit (e.g. a gasless approval to the migrator ahead of migrate) can be silently invalidated if a delegateBySig they also signed at the same nonce is submitted first, forcing the user to re-sign before the approval/deposit can proceed. Impact 2: A user's pre-signed delegateBySig (gasless governance delegation) can be invalidated by a permit consuming the shared nonce first, so the delegation reverts and must be re-signed – a nuisance that is more pointed around a time-sensitive governance snapshot. Impact 3: Because both entrypoints are permissionless, a relay or mempool observer that holds a user's two same-nonce signatures can deterministically choose which one to honor and which to brick – a targeted, repeatable-per-signature griefing of the user's chosen action. All three impacts are recoverable (re-sign against the current nonce) with no fund loss. Around a governance snapshot, front-running a pending delegateBySig with the user's own permit can push the delegation past the snapshot timepoint, costing the user that round's voting weight (recoverable for future rounds).
Recommendations	Consider creating documentation for this behavior so users are aware of it.
Comments / Resolution	

sPRL2V2

The **sPRL2V2** contract is the second version of the **sPRL2** contract and follows the exact same mechanics but omits the AURA deposit from the BPT. The reason for this change is the sunseting of the Aura protocol.

The exact changes can be found here:

- <https://www.diffchecker.com/yoOzwHr6/>

Therefore, users can mint sPRL2V2 with BPT tokens directly or via providing PRL/WETH (ETH) which then routes through the Balancer liquidity addition and creates the BPT which simply remains sitting in the sPRL2V2 contract. For withdrawals, it allows for withdrawing BPT or withdrawing PRL/WETH. However, withdrawals are extended with the mechanism described in TimelockPenaltyERC20.

An **emergencyWithdraw** function is exposed which allows for withdrawals when the contract is paused, without any fee/delay.

Appendix: Balancer Interaction

The contract interacts with the BalancerV3Router and uses it to add and remove liquidity via both functions [addLiquidityProportional/removeLiquidityProportional]. The Router implementation can be found at:

<https://etherscan.io/address/0x5C6fb490BDFD3246EB0bB062c168DeCAF4bD9FDd#code>

Privileged Functions

- none

Invariants

INV 1: emergencyWithdraw is only callable if the contract is paused

INV 2: deposits can only be executed if the contract is not paused

INV 3: Any unconsumed amount during deposits [PRL/WETH(ETH)] must be refunded to the caller

Issue_02	Duplicate symbol during migration process can be confusing
Severity	Medium
Description	During the migration period, both tokens are live (sPRL2 and sPRL2V2). Both tokens share the same symbol (sPRL2), which can then result in consumer down-stream effects if there are any consumers which rely on the symbol accuracy. This can be specifically problematic if the source of truth is related to the deployment address as most likely both addresses point to the DAO. This is distinctly different from a fake sPRL2 token due to the above explained condition. Furthermore, if the reward-merkle distributor's off-chain input uses token symbol/name to select eligible staking balances, users may be excluded or double-counted during the migration window
Recommendations	Consider changing the symbol to sPRL2V2.
Comments / Resolution	

Issue_03	Extra rewards cannot be withdrawn
Severity	Medium
Description	Currently, the BPT remains idle in the sPRL2V2 contract. If there are ever any rewards, such as MERKL rewards, these can be claimed onBehalf to the sPRL2V2 contract (this is usually the way how MERKL rewards work), which then makes these permanently locked in the contract without the ability to withdraw them.
Recommendations	Consider implementing a claim function.
Comments / Resolution	

Issue_04	Missing native ETH exit
Severity	Low
Description	Currently, the <code>sPRL2V2</code> contract allows for depositing via PRL/WETH and PRL/ETH. An inconsistency is present because it does only allow exiting via PRL/WETH and not via PRL/ETH (missing native ETH exit). This limits the UX and forces users to do a native ETH unwrap manually, if they require native ETH instead of WETH.
Recommendations	Consider whether it is desired to have an ETH exit. If yes, we recommend simply adding an additional <code>(isETH)</code> boolean to the <code>withdrawPRLAndWETH</code> function which then unwraps WETH to ETH and executes an ETH transfer. It should be ensured that in this scenario, no additional WETH transfer is executed.
Comments / Resolution	

Issue_05	<code>requestWithdraw</code> can be invoked with zero amount
Severity	Low
Description	Currently, the <code>requestWithdraw</code> function can be called with <code>amount = 0</code>. This will produce an empty request (which can even be cancelled again). This interaction is not required during the normal business operation and as such only produces unnecessary tail risk or a precondition for potential other invalid states.
Recommendations	Consider ensuring that this function cannot be called with <code>amount = 0</code> .
Comments / Resolution	

Issue_06	<code>feeReceiver</code> coupled exits can produce failure states
Severity	Low
Description	Currently, normal exits via <code>withdrawPRLandWETH</code> require an active withdrawal request via <code>requestWithdraw</code> . It can be executed even if the deadline has not passed and will then simply take a fee in BPT. While BPT does not appear to be a token with a blacklist, it is theoretically possible for the <code>feeReceiver</code> being <code>address(0)</code> , which would then result in a revert and as such results in a DoS of all premature withdrawals.
Recommendations	Consider making sure that the <code>feeReceiver</code> is never set to <code>address(0)</code> .
Comments / Resolution	

Issue_07	minimum amounts must be calibrated based on post-fee amount
Severity	Low
Description	Currently, minimum amounts can be provided during every withdrawal [PRL/WETH]. These minimum amounts must be based on the post-slashed BPT amount. Due to the fact that the fee decreases over time, it is possible that a transaction at $t=1$ is only executed at $t=10$ while having the minimum amounts calculated corresponding to $t=1$. This means the minimum amounts are actually less strict than they should be, because the fee is now decreased which will naturally increase the received amounts. Impact 1: A delayed <code>withdrawPRLAndWeth()</code> transaction can execute with weaker-than-intended slippage protection because the min amounts were calculated for a smaller post-fee BPT amount. Impact 2: The user can burn more net BPT through Balancer than expected while still accepting the stale minimum outputs originally calculated for an earlier, higher-fee state.
Recommendations	Consider adding documentation which describes this scenario.
Comments / Resolution	

Issue_08	Post-slash withdrawal amounts can revert due to Balancer V3 minimum trade amount
Severity	Informational
Description	<code>sPRL2V2.withdrawPRLAndWeth()</code> lets users settle pending withdrawal requests into PRL/WETH instead of raw BPT. The function first consumes request IDs through <code>_withdrawMultiple()</code>, which calculates the current slash amount from the timelock penalty and returns the user's net BPT amount plus the slashed BPT amount. <code>_exitPool()</code> then transfers the slashed BPT to <code>feeReceiver</code> and calls Balancer V3 <code>removeLiquidityProportional()</code> with only the user's net BPT amount. This flow is important because the protocol offers two same-goal withdrawal routes: <code>withdrawBPT()</code> returns net BPT directly, while <code>withdrawPRLAndWeth()</code> performs an additional Balancer exit. The second route inherits Balancer's external minimum trade constraints. The root cause is that <code>withdrawPRLAndWeth()</code> does not account for Balancer V3's minimum trade amount before calling <code>removeLiquidityProportional()</code>. The contract passes <code>_bptAmount</code>, which is already reduced by any early-withdrawal slash, into: <pre>BALANCER_ROUTER.removeLiquidityProportional(address(BPT), _bptAmount, minAmountsOut, false, "").</pre> If <code>_bptAmount</code> is sufficiently small, or if the proportional PRL/WETH outputs computed from that BPT amount are sufficiently small after scaling, Balancer can revert through its minimum-trade-amount guard. Balancer documents this guard as applying to add/remove liquidity as well as swaps, and the Vault's remove-liquidity implementation validates both the BPT amount in and the scaled token outputs. The issue does not corrupt request state because if Balancer reverts, the whole transaction reverts, including request status changes, <code>unlockingAmount</code> updates, and the prior slashed-BPT transfer.

Recommendations	Consider adding documentation which describes this scenario.
Comments / Resolution	

Issue_09	Pending requests on sPRL2 can bypass waiting period
Severity	Informational
Description	Any pending requests on sPRL2 are actively waiting until the waiting period has passed to claim without any fees. Due to the migration process and the pausing of the contract, users can now cancel their requests and execute an emergency withdrawal without paying any fee. The only difference lies between the receipt of BPT compared to the receipt of PRL/WETH when no emergency withdrawal is used.
Recommendations	This is rather a note than an issue and should be acknowledged.
Comments / Resolution	

Issue_10	<code>balanceOf</code> is vulnerable to manipulation
Severity	Informational
Description	The <code>requestWithdraw</code> function burns shares (decreases balance) and the <code>cancelWithdrawalRequests</code> function mints shares (increases balance) again. This can all be done at no cost (besides gas). If there is ever any logic (ie. governance voting mechanism) which depends on the integrity of <code>balanceOf</code>, this can be manipulated.
Recommendations	Consider keeping this in mind
Comments / Resolution	

Issue_11	Voting power denomination is fluctuating
Severity	Informational
Description	Within the same repository, a <code>sPRL1</code> contract is existing which allows users to stake PRL tokens and receive <code>sPRL1</code>. The VP of <code>sPRL1</code> is equivalent to 1 unit and the VP of <code>sPRL2V2</code> is equivalent to 2.5 units. Since <code>sPRL2V2</code> represents BPT and this can fluctuate in the underlying ratio, the VP is therefore fluctuating. Furthermore, the 2.5 unit equivalent is not enforced
Recommendations	Consider keeping this in mind
Comments / Resolution	

Issue_12	Address-derived <code>isReversedBalancerPair</code> creates an unvalidated balancer token-order dependency
Severity	Informational
Description	<code>sPRL2V2</code> allows users to enter the vault either by depositing BPT directly or by depositing PRL with WETH/native ETH, which the contract converts into BPT through the Balancer V3 router. The Balancer join and exit paths require token arrays to be supplied in the pool's registered token order. <code>sPRL2V2</code> does not query the Balancer pool or vault for that order. Instead, it stores <code>isReversedBalancerPair = address(WETH) > address(PRL)</code> in the constructor and uses that boolean to encode PRL/WETH arrays for <code>_joinPool()</code> and <code>_exitPool()</code>. The affected flows are <code>depositPRLAndWeth()</code>, <code>depositPRLAndEth()</code>, and <code>withdrawPRLAndWeth()</code>. Direct <code>depositBPT()</code>, <code>withdrawBPT()</code>, and <code>sPRL2Migrator.migrate()</code> do not depend on Balancer token-array ordering. The root cause is that the contract assumes Balancer token registration order can be inferred from the relative addresses of <code>PRL</code> and <code>WETH</code>. That assumption is not enforced by <code>sPRL2V2</code> and is not guaranteed by the local code. The constructor accepts arbitrary <code>balancerBPT</code>, <code>prl</code>, and <code>weth</code> addresses through <code>BPTConfigParams</code>, but it does not verify that the configured BPT is a two-token PRL/WETH Balancer pool or that the pool's registered token order equals the order implied by <code>address(WETH) > address(PRL)</code>. If the configured pool order matches the derived order, the flow is correct. For the current mainnet PRL/WETH BPT, the pool creation input lists PRL first and WETH second, and the address comparison also causes the contract to encode <code>[PRL, WETH]</code>, so the current reviewed deployment is not broken. If the configured pool order differs, all PRL/WETH array parameters are encoded in the wrong slots. In <code>_joinPool()</code>, <code>_maxPrlAmount</code> can be submitted as the WETH maximum and <code>_maxEthAmount</code> as the PRL maximum. In <code>_exitPool()</code>, <code>_minPrlAmount</code> and <code>_minWethAmount</code> can be

	mapped to the wrong token slots, and returned <code>amountsOut</code> can be decoded into the wrong local variables. Existing user slippage parameters reduce the chance of silent value loss because Balancer should revert when the wrong token maxima/minima cannot satisfy the operation. However, the contract still has no integrity check that prevents deployment with a BPT whose token order does not match the derived boolean.
Recommendations	Consider acknowledging this note.
Comments / Resolution	

Issue_13	findUnlockingIds returns empty array in special scenario
Severity	Informational
Description	<code>TimeLockPenaltyERC20</code> stores withdrawal requests in <code>userVsWithdrawals[user][requestId]</code>, with each user's next request ID tracked by <code>userVsNextID[user]</code>. The request IDs are required to execute <code>withdrawBPT()</code>, <code>withdrawPRLAndWeth()</code>, or <code>cancelWithdrawalRequests()</code>. To help frontends and scripts discover active requests, the contract exposes <code>findUnlockingIds(address _user, uint256 _start, bool _startFromEnd, uint16 _countToCheck)</code>. This helper is inherited by both legacy <code>sPRL2</code> and new <code>sPRL2V2</code>, so the behavior affects old withdrawal request discovery, V2 withdrawal request discovery, and migration-window tooling that may need to identify old pending requests. The root cause is the range conversion when <code>_startFromEnd</code> is true. The function first loads <code>nextId = userVsNextID[_user]</code>. If <code>_startFromEnd</code> is true, it rewrites <code>_start</code> as: <pre><code>_start = nextId - _start.</code></pre> For <code>_start = 0</code>, this becomes <code>_start = nextId</code>. The function then computes: <pre><code>end = _start + _countToCheck</code></pre> and clamps <code>end</code> down to <code>nextId</code>. This results in: <pre><code>start = nextId end = nextId</code></pre> so the allocated search range has length zero and the loop checks no IDs. This means a natural query such as "start from the end, check the latest N request IDs" using <code>_start = 0</code>, <code>_startFromEnd = true</code>, and <code>_countToCheck = N</code> returns an empty array, even when the user has

	active UNLOCKING requests. The issue does not affect the underlying request state. Direct request IDs still work, emitted events still contain IDs, and the mappings remain accessible. The bug is in helper discoverability.
Recommendations	Consider keeping this in mind
Comments / Resolution	

Issue_14	uint160 casting can truncate to lower bits
Severity	Informational
Description	Within the deposit flow, the approval amounts are casted as uint160: <pre><i>PERMIT2.approve(address[PRL], address[BALANCER_ROUTER], uint160[_maxPrlAmount], 0);</i></pre> This can theoretically silently truncate the requested amount. However, since the supply of both tokens is never meant to reach this value, this is just a consistency note. Example: <pre>> maxAmountsIn = 2^160 + 100 > Permit2 allowance = 100</pre>
Recommendations	Consider keeping this in mind
Comments / Resolution	

Issue_15	Theoretical supply limitation
Severity	Informational
Description	<code>sPRL2V2</code> inherits <code>TimeLockPenaltyERC20</code>, which in turn inherits OpenZeppelin <code>ERC20Votes</code>. The token is minted whenever a user deposits BPT, deposits PRL/WETH or PRL/ETH through Balancer, migrates old <code>sPRL2</code> through <code>sPRL2Migrator</code>, or cancels an active withdrawal request. The migration path is especially relevant because <code>sPRL2Migrator.migrate()</code> withdraws BPT from legacy <code>sPRL2</code>, deposits the same BPT amount into <code>sPRL2V2</code>, and relies on <code>sPRL2V2.depositBPT()</code> minting V2 shares to the migrator before transferring them to the user. Because the token is vote-enabled, every mint also updates voting units and passes through <code>ERC20Votes</code> supply-safety logic. The root cause is the inherited <code>ERC20Votes</code> cap on token supply. OpenZeppelin <code>ERC20Votes</code> stores voting checkpoints in a compressed format and rejects minting if the resulting <code>totalSupply()</code> exceeds <code>type(uint208).max</code>. <code>TimeLockPenaltyERC20._deposit()</code> calls <code>_mint(msg.sender, amount)</code>. In <code>sPRL2V2</code>, this is reached by direct BPT deposits, Balancer deposits, and migrator deposits. <code>cancelWithdrawalRequests()</code> also calls <code>_mint()</code> when restoring previously burned shares. If V2 <code>totalSupply()</code> were already close to <code>type(uint208).max</code>, a migration or deposit that would push supply above that cap would revert. Similarly, if a user had an active withdrawal request and other deposits refilled supply near the cap after the user's burn, cancelling the request could revert because the remint would exceed the <code>ERC20Votes</code> cap. The existing logic does not locally check this cap before accepting BPT or before entering migration. However, the revert is atomic: failed

	deposits or migrations do not leave BPT moved without shares, and failed cancellations do not corrupt unlockingAmount .
Recommendations	Consider keeping this in mind
Comments / Resolution	

sPRL2Migrator

The **sPRL2Migrator** contract is responsible for the user-triggered migration from sPRL2 to sPRL2V2 and the migration flow is as follows:

- a: Transfer in of user's sPRL2 token
- b: Execution of emergencyWithdraw from sPRL2 and receipt of BPT
- c: Deposit of BPT into sPRL2V2 and receipt of sPRL2V2
- d: Transfer of sPRL2V2 to user

Privileged Functions

- none

Issue_16	Event: Migrated deposits are attributed to migrator instead of actual owner
Severity	Informational
Description	During the migration process, the migrator calls sPRL2V2.depositBPT which grants these funds first to the migrator and only then the migrator transfers them out to the actual owner. This will produce the following event: <pre><i>emit Deposited(msg.sender, _amount);</i></pre> Which uses msg.sender = migrator and as such it can be misleading for third parties.
Recommendations	Consider implementing documentation which highlights this behavior.
Comments / Resolution	